



THE UNIVERSITY OF  
MELBOURNE

# COMP90050 Advanced Database Systems

## Semester 1, 2026

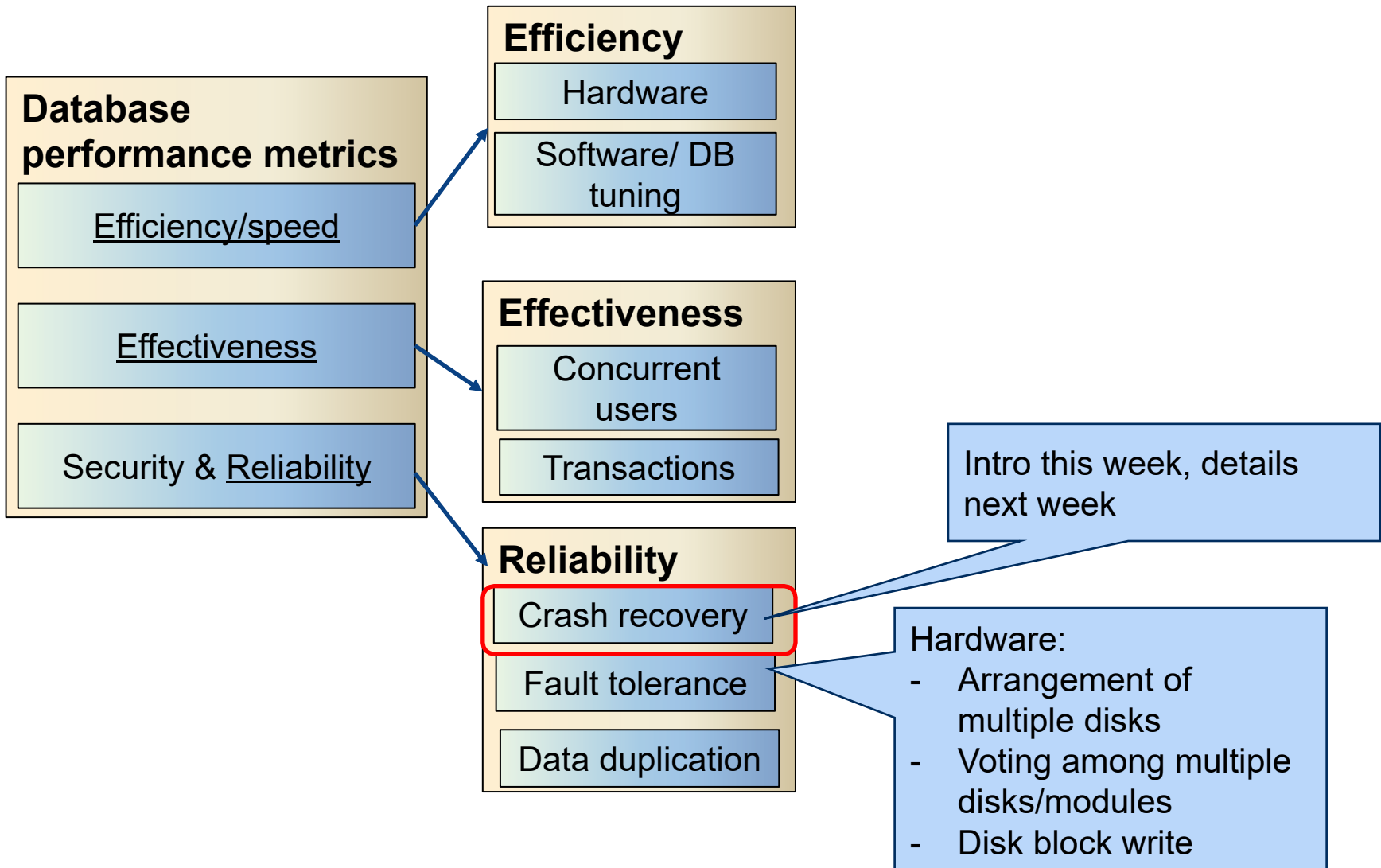
Lecturer: Farhana Choudhury (PhD)

Live lecture – Week 9





# Core Concepts of Database management system

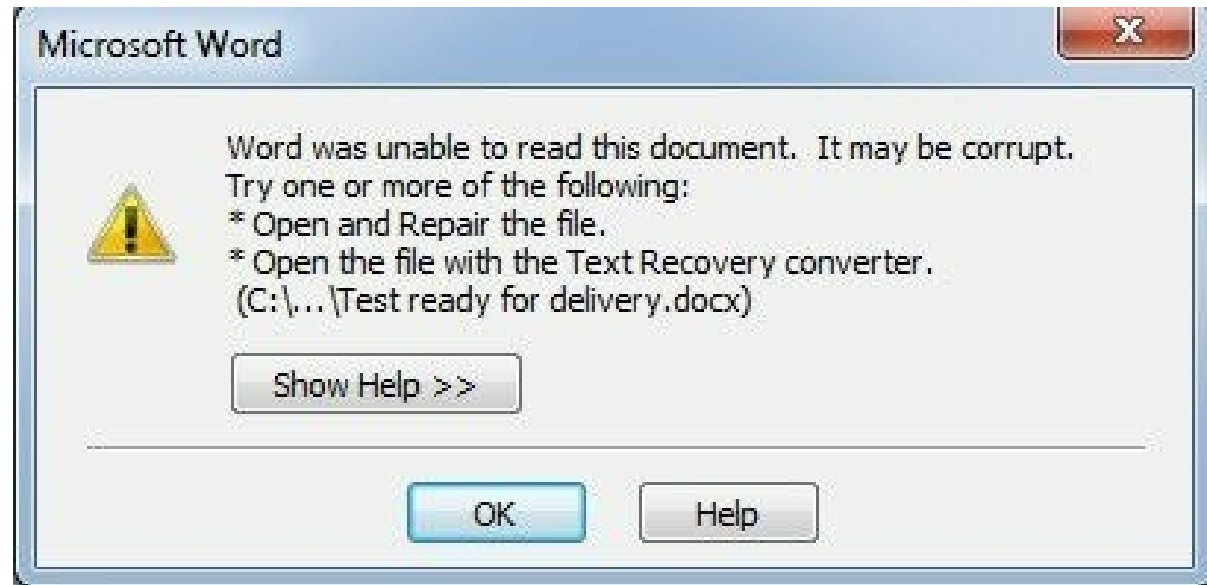


# Disk writes

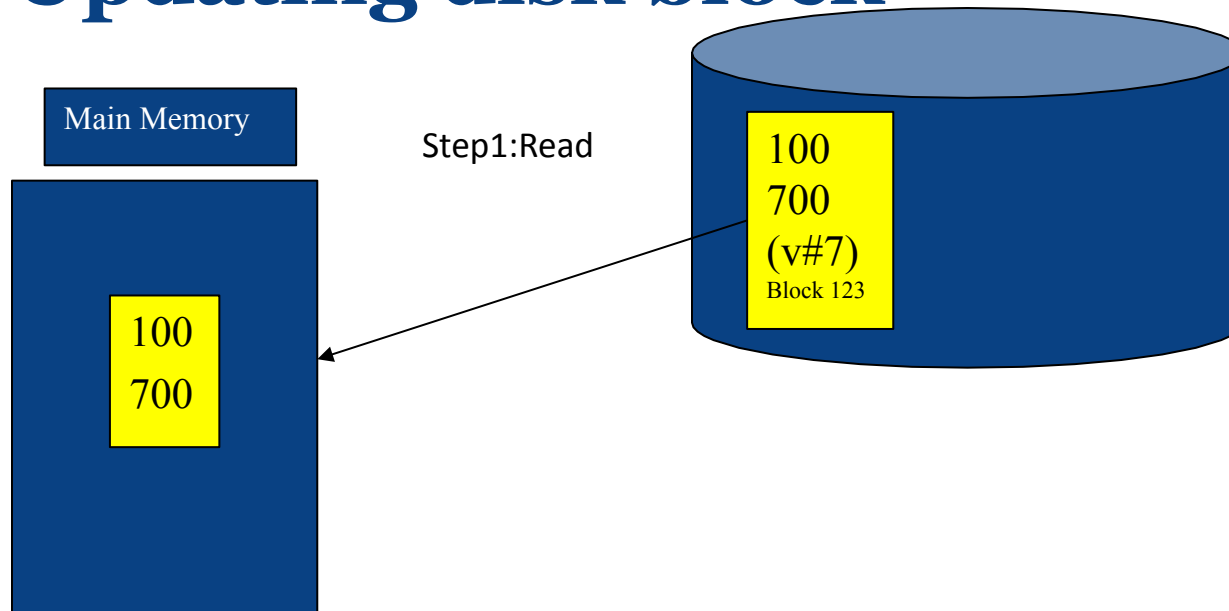
## Disk writes for consistency:

Either entire block is written **correctly** on disk or the contents of the block is unchanged. To achieve disk write consistency we can do –

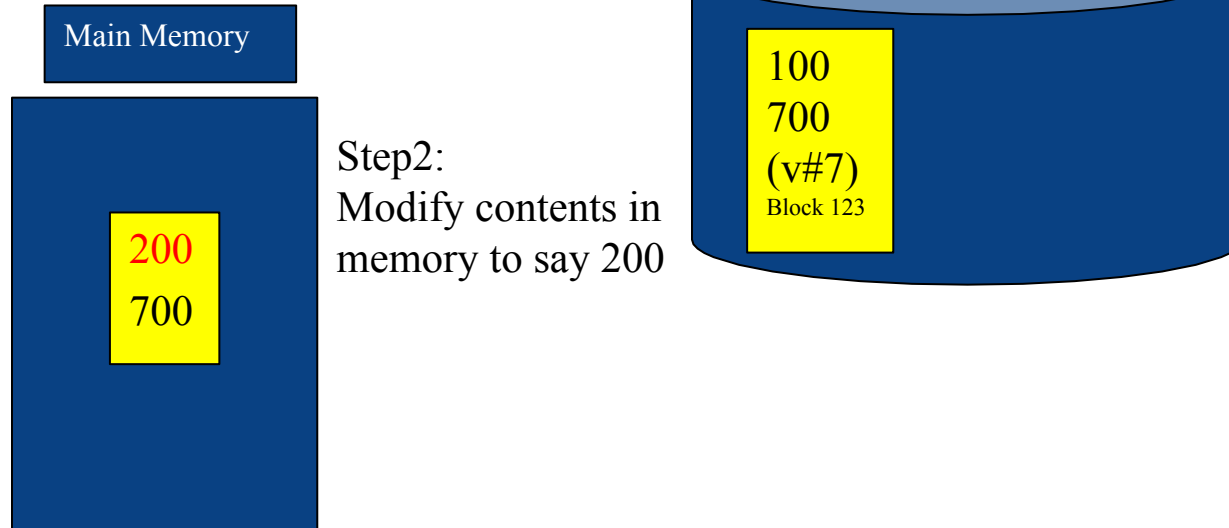
- **Duplex** write
- **Logged** write



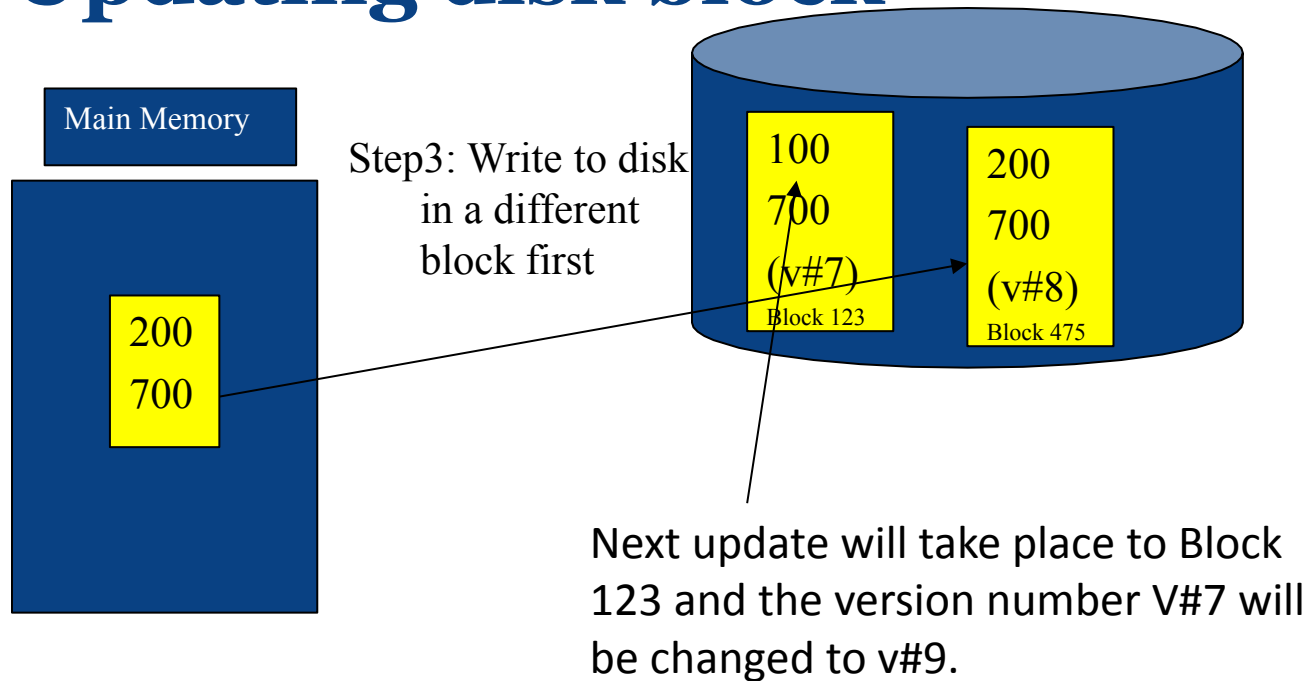
# Updating disk block



# Updating disk block



# Updating disk block



(Two different physical disks can be used for duplex writes as well)

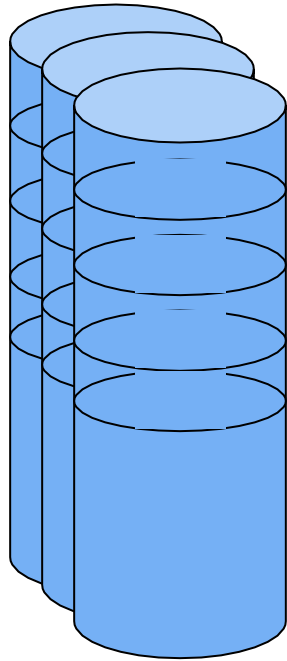


***Logged*** write- similar to duplex write, except one of the writes goes to a log. This method is very efficient if the changes to a block are small.

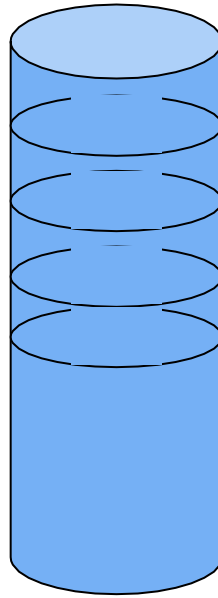
The first one of the writes goes to a log. The second overwrites the old regular data block. In short, **all modifications need to be logged before they are applied.**

So **if a failure occurs, system knows where we are left** to take proper action, i.e., even during a single block write.

# Some configurations



Disk for  
data blocks

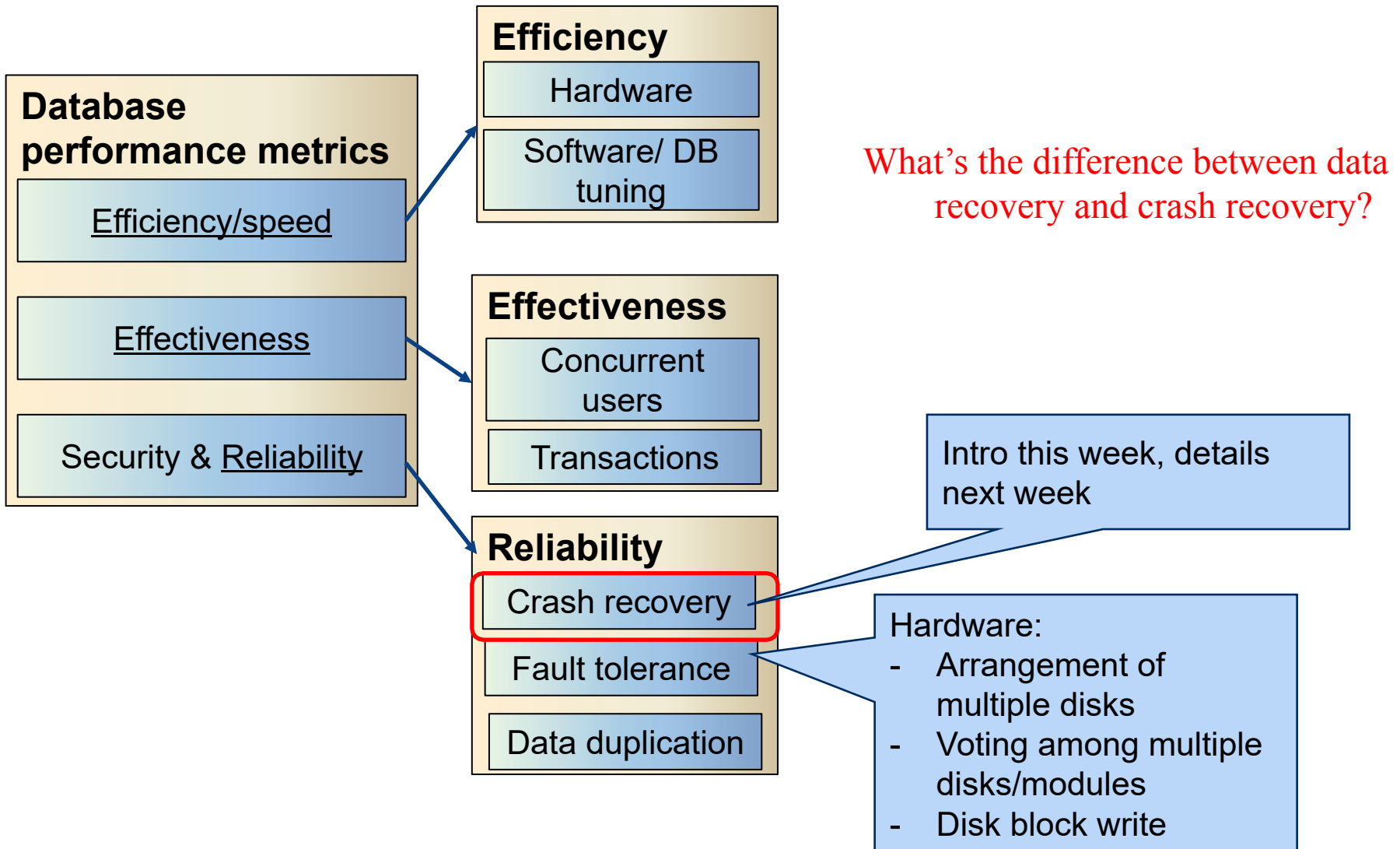


Disk for logs





# Core Concepts of Database management system





# Crash recovery

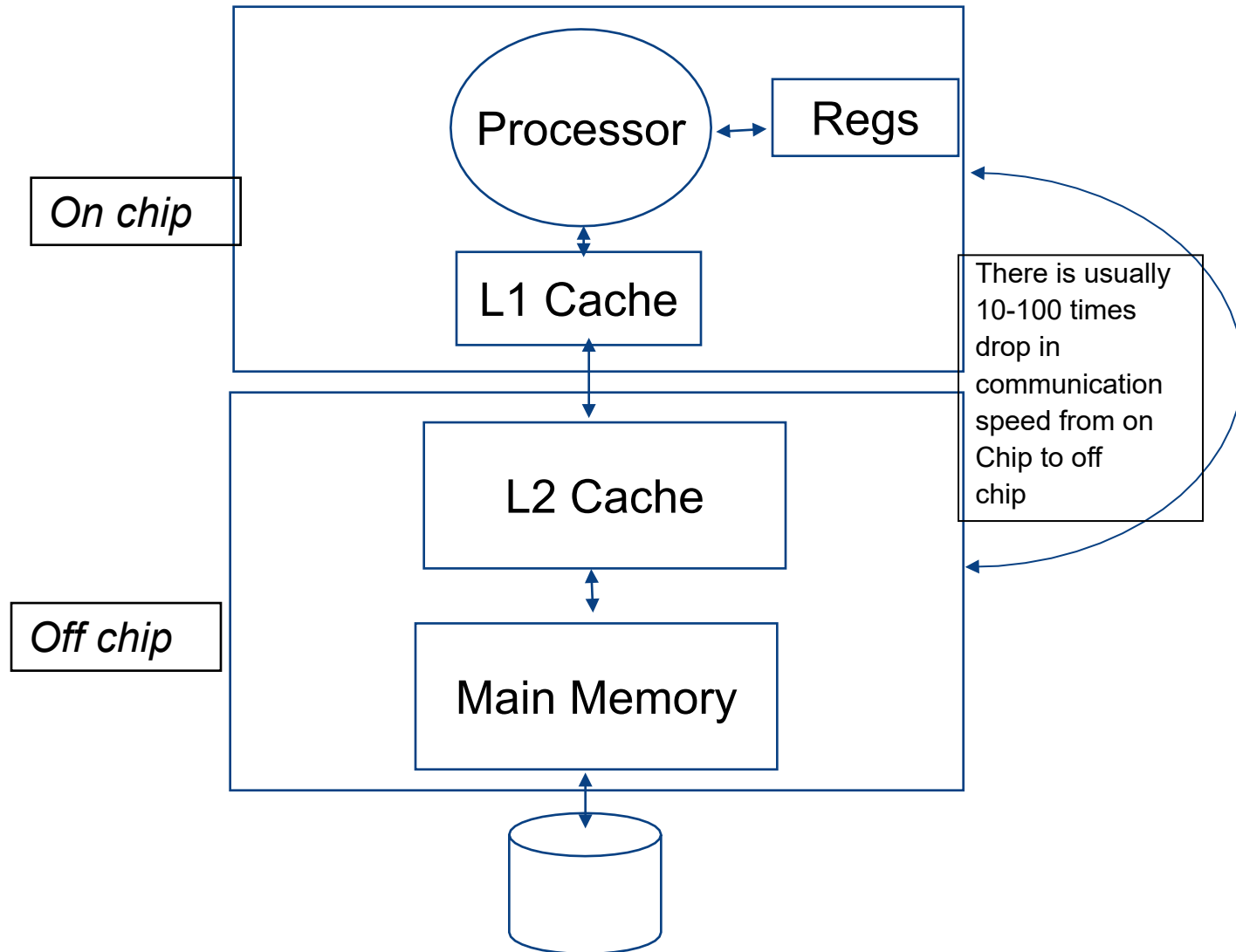
What needs to be recovered if a crash happens?

- Transaction committed and its changes are made durable - good!
- Transaction aborted and its changes are undone and made durable – good!
- If not durable – what additional information are needed to recover them?
- If a transaction was running – cleanup

So at first, we need to know what happens during the usual execution of a system.

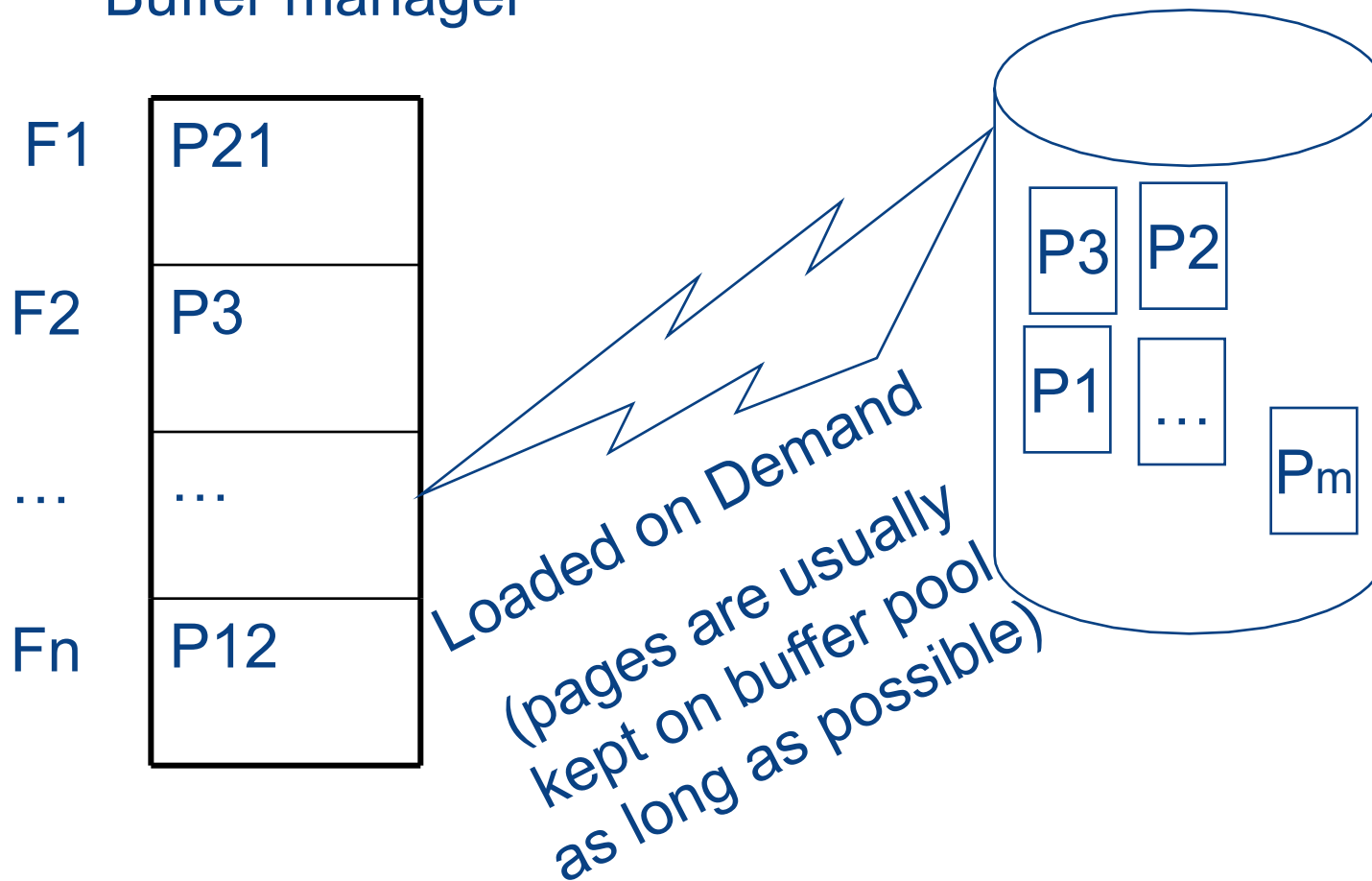
# Memory hierarchy – week 1's lecture

## What makes some data non-durable?



Performance reason - disk read/writes are time consuming

## Buffer manager





# Combining this knowledge with other topics we learnt before

Let's assume that a bitmap index is in memory while the hard disk became unavailable

If the system is designed to still run\* while the disk is being recovered,  
Can we still get answers for queries “How many people are in income level L1”?

| Record Num | Name   | State | Income_level |
|------------|--------|-------|--------------|
| 0          | John   | VIC   | L1           |
| 1          | Diana  | NSW   | L2           |
| 2          | Xiaolu | WA    | L1           |
| 3          | Anil   | VIC   | L4           |
| 4          | Peter  | NSW   | L3           |

| Bitmap for income level | Income_level |
|-------------------------|--------------|
| L1                      | 1 0 1 0 0    |
| L2                      | 0 1 0 0 0    |
| L3                      | 0 0 0 0 1    |
| L4                      | 0 0 0 1 0    |
| L5                      | 0 0 0 0 0    |

**Time for a poll -** [Pollev.com/farhanachoud585](https://pollev.com/farhanachoud585)



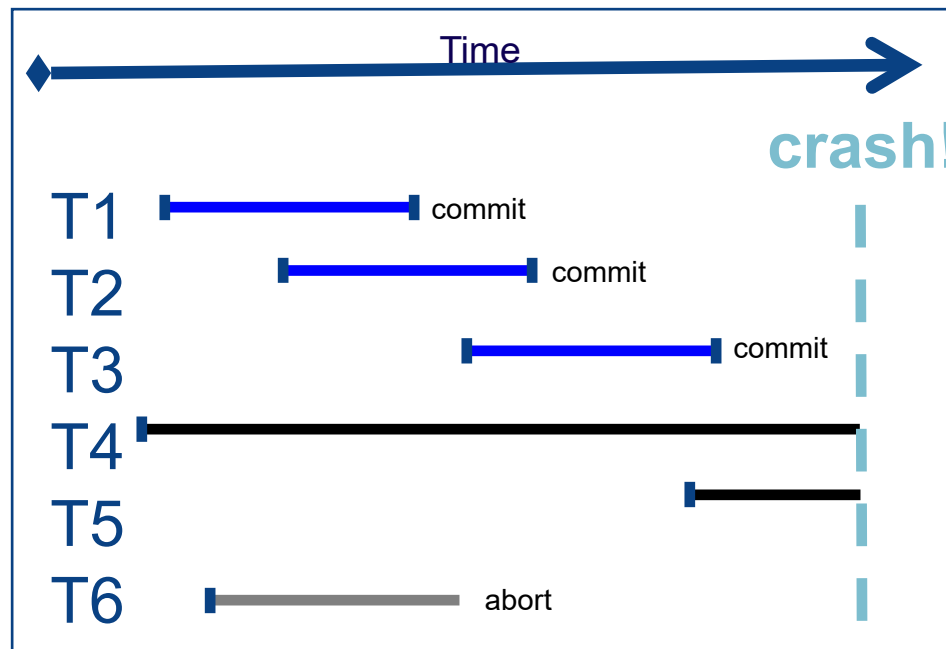


## **Non-examinable resource:**

A good reading (non-examinable) - Here's a good resource on the basics of buffer management if you are really interested. Note that, this contents in this reading link (that are not covered by the lecture or tutorial) will not be examined -

[https://www.researchgate.net/publication/220225054\\_Principles\\_of\\_Data\\_base\\_Buffer\\_Management](https://www.researchgate.net/publication/220225054_Principles_of_Data_base_Buffer_Management)

# What we want if a crash happens



ARIES for crash recovery – most DBMSs use this algorithm (or its variant)

# Crash recovery intro - summary

What needs to be recovered if a crash happens?

- Has it been made durable (and should be durable) - good!
- If not durable – what additional information are needed to recover them?

Data pages in the buffer pool

Detailed steps next week

**Crash manager maintains both durability and atomicity**

The changes by committed transactions – make them durable  
The changes by aborted/running transactions - undo



# *Instance of Log, Transaction and Page tables*

Dirty Page table

| Pag<br>e# | Oldest<br>LSN<br>(Recent<br>LSN) |
|-----------|----------------------------------|
|           |                                  |
|           |                                  |
|           |                                  |

X-table

| Xid | Statu<br>s | Last<br>LSN |
|-----|------------|-------------|
|     |            |             |
|     |            |             |

Log

| Prev<br>LSN | Tid | Type | P<br>ag<br>e | Le<br>ng<br>th | Of<br>fs<br>et | Old<br>Value | New<br>Value |
|-------------|-----|------|--------------|----------------|----------------|--------------|--------------|
|             |     |      |              |                |                |              |              |



# Instance of Log, Transaction and Page tables

Dirty Page table

| Pag e# | Oldest LSN (Recent LSN) |
|--------|-------------------------|
| P5     |                         |
|        |                         |
|        |                         |

What happens if T1 needs to read from page P1?

Log

| Prev LSN | Tid | Type | P age | Le ng th | Of fs et | Old Value | New Value |
|----------|-----|------|-------|----------|----------|-----------|-----------|
|          | T1  | U    | p5    | 4        | 20       | abcd      | ABCD      |

X-table

| Xid | Statu s | Last LSN |
|-----|---------|----------|
| T1  | R       |          |
|     |         |          |

# Instance of Log, Transaction and Page tables

Dirty Page table

| Pag<br>e# | Oldest<br>LSN<br>(Recent<br>LSN) |
|-----------|----------------------------------|
| P5        |                                  |
| P6        |                                  |
|           |                                  |

Log

| Prev<br>LSN | Tid | Type | P<br>ag<br>e | Le<br>ng<br>th | Of<br>fs<br>et | Old<br>Value | New<br>Value |
|-------------|-----|------|--------------|----------------|----------------|--------------|--------------|
|             | T1  | U    | p5           | 4              | 20             | abcd         | ABCD         |
|             | T2  | U    | p6           | 4              | 40             | efgh         | EFGH         |

X-table

| Xid | Statu<br>s | Last<br>LSN |
|-----|------------|-------------|
| T1  | R          |             |
| T2  | R          |             |



# Instance of Log, Transaction and Page tables

Dirty Page table

What if P5 is not in the buffer and dirty page table anymore?

| Page# | Oldest LSN (Recent LSN) |
|-------|-------------------------|
| P5    |                         |
| P6    |                         |
|       |                         |

Log

| Prev LSN | Tid | Type | Page | Length | Offset | Old Value | New Value |
|----------|-----|------|------|--------|--------|-----------|-----------|
|          | T1  | U    | p5   | 4      | 20     | abcd      | ABCD      |
|          | T2  | U    | p6   | 4      | 40     | efgh      | EFGH      |
|          | T2  | U    | p5   | 4      | 80     | jklm      | JKLM      |

X-table

| Xid | Status | Last LSN |
|-----|--------|----------|
| T1  | R      |          |
| T2  | R      |          |



# Crash recovery intro - summary

What needs to be recovered if a crash happens?

- Has it been made durable (and should be durable) - good!
- If not durable – what additional information are needed to recover them?

Data pages in the buffer pool

Detailed steps next week

**Crash manager maintains both durability and atomicity**

The changes by committed transactions – make them durable  
The changes by aborted/running transactions - undo